Pergamon

BibLaTeX-inspired bibliography management for Typst
https://github.com/alexanderkoller/pergamon

v0.8.1, 21 July 2026
Alexander Koller

Contents

1 Introduction

Pergamon is a package for typesetting bibliographies in Typst. It is inspired by BibLaTeX, in that the way in which it typesets bibliographies can be easily customized through Typst code. Like Typst's regular bibliography management model, Pergamon can be configured to use different styles for typesetting references and citations; unlike it, these styles are all defined through Typst code, rather than CSL.

Pergamon has a number of advantages over the builtin Typst bibliographies:

- Pergamon styles are simply pieces of Typst code and can be easily configured or modified.
- The document can be easily split into different `refsections`, each of which can have its own bibliography (similar to Alexandria). Unlike in Alexandria, you do not have to manually add bibliography prefixes to your citations.
- Paper titles can be automatically made into hyperlinks – as in blinky, but much more flexibly and correctly.
- Bibliographies can be filtered, and bibliography entries programmatically highlighted, which is useful e.g. for CVs.
- References retain nonstandard BibTeX fields (unlike in Hayagriva), making it e.g. possible to split bibliographies based on keywords.

I have used Pergamon for a number of nontrivial bibliographic scenarios, but I welcome your comments so that I can make it work more generally. BibLaTeX is a very complex library to replicate, and I would like to prioritize features that people actually care about.

Pergamon was an ancient Greek city state in Asia Minor. Its library was second only to the Library of Alexandria around 200 BC.

2 Example

The following piece of code typesets a bibliography using Pergamon. (You can try out a more complex example yourself: download example.typ from Github; see also the generated PDF.)

```
#import "@preview/pergamon:0.8.1": *

#add-bib-resource(read("bibliography.bib"))

#refsection(style: numeric-style())[
  ... some text here ...
  #cite("bender20:_climb_nlu")

  #print-bibliography()
]
```

This will generate the output shown in Figure 1. Let's go through the different parts of this code one by one.

... some text here ... [1]

References

[1] Emily M. Bender and Alexander Koller. "Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020.

Figure 1: Example bibliography typeset with Pergamon (`numeric` style).

The first relevant function that is called here is `add-bib-resource`. It parses a BibTeX file and adds all the BibTeX entries to Pergamon's internal list of references. Notice that you have to `read` the BibTeX file yourself and pass its contents to `add-bib-resource` as a string. This is because Typst packages can't access files in your working directory for security reasons.

We then create a `refsection`. A `refsection` is a section of Typst content that shares a bibliography. Pergamon tracks the citations within each refsection separately and prints only those references that were cited within the current refsection when you call `print-bibliography`.

The actual citation is generated by the call `cite("bender20:_climb_nlu")`. Pergamon currently does not use the regular Typst citation syntax `@bender20:_climb_nlu` for a number of reasons (see issue #40). Instead you call Pergamon's `cite` function, with the key of the BibTeX entry as a string (not a Typst label). Note that this is not the same as Typst's builtin cite function, which is overwritten by Pergamon.

The call to `refsection` has a parameter `style`, to which we passed `numeric-style()`. This specifies the *style* that should be used to typeset citations and bibliographies in this refsection. *Numeric* is one of the three builtin styles of Pergamon the two others, *alphabetic* and *authoryear*, will be demonstrated below. The call to `print-bibliography` at the end of the refsection typesets all the references that were cited in this refsection; the style controls how exactly it typesets the references.

Observe that `numeric-style()` has an opening and closing bracket after the function name, indicating that we want to use the numeric style with its default parameters. You can customize how Pergamon typesets citations and references by passing different arguments to the styles; modifying the builtin citation and reference styles; and even implementing your own styles from scratch. This is explained below.

3 Pergamon styles

Pergamon is highly configurable with respect to the way that references and citations are rendered. Its defining feature is that the *styles* that control the rendering process are defined as ordinary Typst functions, rather than in CSL.

There are two different types of styles in Pergamon:

- *Reference styles* define how the individual references are typeset in the bibliography.
- *Citation styles* define how citations are typeset in the text.

Pergamon comes with one predefined reference style and three predefined citation styles. We will explain these below, and then we will sketch how to define your own custom styles.

For convenience, a matching reference style and citation style can be packaged into a *style bundle*. When we passed `style: numeric-style()` in the example of Section 2, we specified a style bundle for the refsection. While this section focuses on explaining the reference and citation styles separately, you will probably mostly use style bundles in practice. They are explained in Section 3.6.

3.1 Specifying styles

There are two fundamental ways in which a style can be specified in Pergamon. The first is through a style bundle, as shown in the example in Section 2: You pass a style bundle to the refsection's `style` argument; the citations within this refsection then use this bundle's citation style, and all calls to `print-bibliography` in the refsection use the bundle's reference style. This should be sufficient for most uses of Pergamon.

Alternatively, you can specify the citation and reference styles separately:

```
#import "@preview/pergamon:0.8.1": *

#let style = format-citation-numeric()
#add-bib-resource(read("bibliography.bib"))

#refsection(format-citation: style.format-citation)[
  ... some text here ...
  #cite("bender20:_climb_nlu")

  #print-bibliography(
    format-reference:
      format-reference(reference-label: style.reference-label),
    label-generator: style.label-generator
  )
]
```

This will generate the same output as the original example (as in Figure 1); the code is more complicated, but makes the different components of the style bundle visible. Notice that the refsection is only passed a citation style. The `print-bibliography` function is passed a reference style in the `format-reference` argument. The citation and reference style are connected through the `label-generator`, which generates the labels by which each reference is displayed when it is cited.

3.2 Builtin reference style

The builtin reference style is defined by the `format-reference` function. This reference style aims to replicate the default style of BibLaTeX.

... some text here ... [1]

References

[1] Emily M. Bender and Alexander Koller (2020). Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL).*

Figure 2: Bibliography with the configuration of Section 3.2.

The builtin reference style can currently render all BibLaTeX entry types (article, inproceedings, etc.) except for `set`. These are explained in more detail in Section 2.1.1 of the BibLaTeX documentation. Entry type aliases are resolved as in BibLaTeX (e.g. `software` to `misc`).

The builtin `format-reference` function can be customized by passing arguments to it. These arguments are explained in detail in Section 7.3. As one example, the following arguments will change the output of the example above to look like in Figure 2:

```
#print-bibliography(
  format-reference: format-reference(
    reference-label: style.reference-label,
    print-date-after-authors: true,
    format-quotes: it => it
  ),
  label-generator: style.label-generator
)
```

We passed `true` for the argument `print-date-after-authors`. This moved the year from the end of the reference to just after the authors and put it in brackets.

We also passed the function `it => it` as the `format-quotes` argument. The builtin reference style surrounds all papers that are contained in bigger collections (in this case, a volume of conference proceedings) in quotes by applying the `format-quotes` function. By replacing the default `format-quotes` function with the identity function, we can make the quotes disappear in the output.

3.3 Builtin citation styles

Pergamon comes with three builtin citation styles: *alphabetic*, *numeric*, and *authoryear*. These replicate the BibLaTeX styles of the same names (see e.g. the examples on Overleaf).

Unlike in Typst's regular bibliography mechanism, you write `#cite("key")` to insert a citation into your document when you use Pergamon, *not* `#cite(<key>)`. Pergamon replaces this citation command by a citation string, which depends on the citation style you use.

The difference between the three builtin citation styles is illustrated in Figure 1 (numeric), Figure 3 (alphabetic), and Figure 4 (authoryear). *Numeric* and *alphabetic* both create a label for each bibliography entry; in the case of *numeric*, the label is the position in the bibliography, and in the case of *alphabetic*, it is a unique string consisting of the first characters of the author

... some text here ... [BK20]

References

[BK20] Emily M. Bender and Alexander Koller. "Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020.

Figure 3: Bibliography with the `alphabetic` citation style.

... some text here ... (Bender and Koller 2020)

References

Emily M. Bender and Alexander Koller. "Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020.

Figure 4: Bibliography with the `authoryear` citation style.

names and the year. In both cases, these labels are displayed next to the references and also used as the string to which `#cite(key)` expands. By contrast, *authoryear* does not display any labels next to the references; it expands `#cite(key)` to a string consisting of the last names of the authors and the year.

Some of the citation styles have options that will let you control the appearance of the citations in detail. These are documented in Section 7.4. For instance, to enclose the year in the *authoryear* style in square brackets rather than round ones, you can replace line 4 in the above example with

```
1 #let style = format-citation-authoryear(format-parens: nn(it => [[#it]]))
```

Citation forms

Each citation style offers you different *citation forms* for presenting your citation. These are documented in Table 1. Citation forms are selected using the optional `form` argument to the `cite` function:

```
1 #cite("bender20:_climb_nlu", form: "t")
```

If you do not specify a citation form, the `auto` citation forms will be used. For your convenience, BibLaTeX defines the functions `citet`, `citep`, `citen`, and `citeg`, which all just call `cite` with the respective citation form.

Citation options

You can pass extra named arguments to the Pergamon citation commands `cite`, `citet`, etc. These arguments will be passed as *options* to the citation formatter. For instance, the following citation will render as "(see Bender et al. 2020, p. 3)":

	authoryear	alphabetic	numeric
`auto`	(Bender and Koller 2020)	[BK20]	[1]
p	(Bender and Koller 2020)	–	–
t	Bender and Koller (2020)	–	–
g	Bender and Koller's (2020)	–	–
name	Bender and Koller	–	–
year	2020	–	–
n	Bender and Koller 2020	BK20	1

Table 1: Citation forms.

... some text here ... (Bender and Koller 2020)

References

Emily M. Bender and Alexander Koller. 2020. "Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*.

Figure 5: Bibliography with modified format-functions.

```
1 #cite("bender20:_climb_nlu", prefix: "see", suffix: "p. 3")
```

Currently, the only builtin citation style that supports such arguments is the *authoryear* style. It accepts the `prefix` and `suffix` options, as in the example. You can still pass options to the other citation styles; they will simply ignore them.

3.4 Customizing the reference style

You can deeply customize how the default reference style in Pergamon typesets the individual references. Your first stop should be to look into the parameters of `format-reference` and `print-bibliography`, see Section 7.2 and Section 7.3.

If this is not flexible enough, you can modify the behavior of the default reference style by defining how specific pieces of references are typeset. The default style defines a variety of functions in reference-styles.typ that typeset pieces of the reference; for instance, the function `journal-issue-title` displays the title and issue of a journal and connects them correctly with commas and periods. At the extreme end of the spectrum, the function `driver-X` renders a complete BibTeX entry of type `X` (e.g. `driver-article` renders journal article references). You can use the parameter `format-functions` in `format-reference` (see Section 7.3) to override these formatting functions.

An example is shown in Figure 5; this reference style replicates that of the ACL conferences, which typesets the year after the author, separated by periods. To achieve this format, you can override the formatting function `maybe-with-date` as shown below:

```
#import "@preview/pergamon:0.8.0": *
#let dev = pergamon-dev

#print-bibliography(
  format-reference: format-reference(
    reference-label: style.reference-label,
    print-date-after-authors: true,
    format-functions: (
      "maybe-with-date": (reference, options) => {
        name => {
          periods(
            name,
            (dev.date-with-extradate)(reference, options)
      )}},
  )),
  label-generator: style.label-generator
)
```

This is actually one of the more complex formatting functions. Like all formatting functions, it takes the `reference` and the usual `options` as argument. However, most other formatting functions just return some Typst content. By contrast, the drivers for the entry types call `maybe-with-date` as a function that takes the rendered author name as input and returns content. By default, `maybe-with-date` attaches the year and extradate to the author name, separated by parentheses, if the option `print-date-after-authors` is specified. The code above replaces this default implementation of `maybe-with-date` with a function that takes the author name as argument and attaches the date and extradate with a period. The drivers for the entry types will now call this function instead of the default one.

The formatting functions that can be overridden are exactly those that start with `with-default` in reference-styles.typ. That file will also give you hints on which formatting function you have to override for the effect you want. You can access the other formatting functions through the dictionary `pergamon-dev`, which you can access after importing Pergamon (see the code above). The default date formatter hooks, such as `default-format-date` and `default-format-date-range`, are also exported directly from Pergamon.

3.5 Implementing your own styles from scratch

If the customization options of the `format-functions` parameter are not enough for you, you can also define your own Pergamon style – either a reference style or a citation style or both. It is recommended to look at the default styles in citation-styles.typ and reference-styles.typ to get a clearer picture.

Implementing a reference style amounts to defining a Typst function that can be passed as the `format-reference` argument to `print-bibliography`. Such a function receives arguments

`(index, reference)` containing the zero-based position of the reference in the bibliography and a reference dictionary. A call to your function should return some `content`, which will be displayed in the bibliography.

Reference dictionaries are a central data structure of Pergamon. They represent the information contained in a single bibliography entry and are explained in detail in Section 7.1.

Implementing a citation style is a little more involved, because a citation style consists of three different functions:

- The *label generator* is passed as an argument to `print-bibliography`. It is a function that receives a reference dictionary and the bibliography position as arguments and is expected to return an array of length two. Its first element is the bib entry's *label*; it is stored under the `label` field of the reference dictionary and can contain any information that your style finds useful. The second element is a string summarizing the contents of the label. It is used to recognize when two bib entries have the same label and therefore need an "extradate" to make it unique, e.g. the letter "a" in a citation string like "Smith et al. (2025a)". It is up to your style to ensure that entries with the same label also have the same string summary.
- The *reference labeler* is passed as an argument to `format-reference`. It receives a reference dictionary and the bibliography position as arguments and returns content. If this content is not `none`, it will be typeset as the first column in the bibliography, next to the reference itself. Of the builtin citation styles, the reference labeler of `authoryear` always returns `none` (indicating a one-column bibliography layout), and the other two return their respective labels in square brackets. Pergamon assumes that the reference labeler of a citation style either always returns `none` or never returns `none`, making for a consistent number of columns.
- The *citation formatter* is passed as an argument to `refsection`. It is responsible for formatting the actual citations into content that is inserted into the document text. The citation formatter receives three arguments: an array of citation specifications, a citation form (cf. "Citation forms" in Section 3.3), and an options dictionary (cf. "Citation options" in Section 3.3). See the `format-citation` argument of the `refsection` function in Section 7.2 for details on the citation specifications.

Note that the label information that the label generator produces will be stored in the `label` field of the reference dictionary. When the reference labeler and the citation formatter are called, the `label` information will still be available, allowing you to precompute any information you find useful.

3.6 Style bundles

A style bundle is a dictionary that packages a reference style, a citation formatter, and a label generator:

```
1 (
2   citation-style: (citation formatter),
3   reference-style: (reference style),
4   label-generator: (label generator)
5 )
```

The citation style also contains a reference labeler, which is used in the construction of the reference style.

Pergamon comes with three builtin style bundles, each of which combines the default reference style with one of the three builtin citation styles:

citation style	style bundle
`format-citation-numeric`	`numeric-style`
`format-citation-alphabetic`	`alphabetic-style`
`format-citation-authoryear`	`authoryear-style`

You can package your own citation and reference styles into a style bundle using the `build-style` function, cf. Section 7.5.

The functions `numeric-style` etc. also permit you to pass arguments to the reference and citation styles they bundle, using their `reference` and `reference` arguments. For instance, recall that we passed arguments `print-date-after-authors` and `format-quotes` to the reference style in Section 3.2. We can write this example more succinctly by passing arguments through the reference style:

```
#let style = numeric-style(
  reference: (
    print-date-after-authors: true,
    format-quotes: it => it
  )
)

#refsection(style: style)[
  ... some text here ...
  #cite("bender20:_climb_nlu")

  #print-bibliography()
]
```

4 Advanced usage

4.1 Multiple refsections

A Pergamon document consists of one or more `refsections`. Each refsection is a segment of the document that shares a bibliography: Pergamon collects all citations within each refsection and prints them in the refsection's bibliography. This allows you to have multiple bibliographies per document, as in the well-known Alexandria package.

Every refsection in a document has a unique identifier that distinguishes it from the other refsections. These identifiers are prepended to the keys of the bibliography entries in every

citation. You still write `#cite("key")` in your document, with the same key that you also use in your BibTeX file; in a refsection with identifier `id`, Pergamon automatically converts this into a reference to `id-key`. The only situation where you will notice that the keys were modified is when a citation is undefined; in this case, Typst will warn you that the reference `id-key` couldn't be resolved, rather than `key`.

You can specify the refsection identifier yourself by passing it as the `id` argument to the `refsection` function. But typical usage will be to not specify an explicit `id` argument and let Pergamon assign a unique identifier automatically. In this case, the first refsection in the document will have the identifier `none`, and the subsequent ones will be names `"ref1"`, `"ref2"`, and so on. When the identifier is `none`, Pergamon simply uses the `key` itself as the label, rather than prepending it with an identifier string. For the frequent use case where the document has only one refsection, this will make error messages easier to read.

You may use `print-bibliography` only inside a `refsection`. If your document has only a single refsection, you can configure it through a document show rule, like this:

```
#show: refsection.with(style: numeric-style())

#add-bib-resource(read("bibliography.bib"))
#cite("bender20:_climb_nlu")
#print-bibliography()
```

4.2 Styling the bibliography

A Pergamon bibliography is displayed as a grid, with one row per bibliography entry. The grid has one or two columns, depending on whether a first column is needed to display a label or not.

You can style this grid by passing a dictionary in the `grid-style` argument of `print-bibliography`. The values in this dictionary will be used to overwrite the default values.

In addition, the individual entries in the bibliography are typeset as paragraphs, one per bib entry. You can style these paragraphs through `set par` rules, e.g. to give the entries a hanging indentation.

4.3 Styling individual references

The default reference style of Pergamon gives you fine-grained control over the way the individual fields of a reference are rendered. A *field formatter* is a function with parameters `(value, reference, field, options, style)`, where `field` is the name and `value` is the value of a field in the reference; `reference` is the entire reference dictionary; `options` are the options that were passed to the reference style; and `style` is an optional style specification for the field. The field formatter is expected to return content representing the field's value.

You can override the formatters for specific fields by using the `format-fields` parameter of `format-reference`. The argument should be a dictionary that maps field names to field

Emily M. Bender and **Alexander Koller**. "Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020.

Figure 6: Reference with highlighted author.

formatters. One use of this mechanism is to highlight specific authors in a reference. For instance, to highlight my name in a reference, I could use the following call:

```
#format-reference(
  format-fields: (
    "author": (dffmt, value, reference, field, options, style) => {
      let formatted-names = value.map(d => {
        let highlighted = (d.family == "Koller")
        let name = format-name(d, format: "{given} {family}")
        if highlighted { strong(name) } else { name }
      })

      concatenate-names(formatted-names, maxnames: 999)
    }
  )
)
```

This will produce output as in Figure 6.

Another effect that can be achieved by overriding field formatters is to change the presentation of the volume and number of the journal in which an article appears. Here's how the default presentation "VOL.NUM" can be replaced with "vol. VOL, no. NUM":

```
#format-reference(
  volume-number-separator: ", ",
  format-fields: (
    "volume": (dffmt, value, reference, field, options, style) => {
      if reference.entry_type == "article" {
        [vol. #value]
      } else {
        dffmt(value, reference, field, options, style)
      }
    },

    "number": (dffmt, value, reference, field, options, style) => {
      if reference.entry_type == "article" {
        [no. #value]
      } else {
        dffmt(value, reference, field, options, style)
      }
    },
  )
)
```

Note that this implementation makes use of the `dffmt` argument, which receives the default implementation of the field formatters for volume and number, respectively, so that we can delegate the formatting of the field for references that are not journal articles.

One special case that is not covered by field formatters arises in the case of subtitles. In BibLaTeX, the titles of journals, books, special issues, and multi-volume books can have optional subtitles. If both are specified, Pergamon concatenates the title and subtitle with the `subtitlepunct` argument, e.g. to "Title: Subtitle". It then applies a formatting function to the entire concatenated title and subtitle; for instance, `format-journaltitle` defaults to setting the title and subtitle in italics. You can override the default behavior by passing your own functions in these arguments.

4.4 Sorting the bibliography

You can furthermore control the order in which references are presented in the bibliography. To this end, you can pass a sorting string in the `sorting` argument of `print-bibliography`. You can also reverse the order in which the references are displayed with the `reversed` parameter. See the documentation of this argument in Section 7 for details.

4.5 Styling the citations

Citations in Pergamon are link elements that point to the location where the reference is typeset in the bibliography. They can be styled using show rules. However, it is not entirely trivial to distinguish a `link` element that represents a Pergamon citation from any other `link` element (referring e.g. to a website). Pergamon therefore provides a function `if-citation` which will make this distinction for you. The following piece of code typesets all Pergamon citations in blue:

```
#show link: it => if-citation(it, value => {
  set text(fill: blue)
  it
})
```

The `value` argument contains metadata about the citation; `value.reference` is the reference dictionary (see Section 7.1). You can use this information to style citations conditionally. For instance, in order to typeset all citations to my own papers in green and all other citations in blue, I could write:

```
#show link: it => if-citation(it, value => {
  if "Koller" in family-names(value.reference.fields.parsed-author) {
    set text(fill: green)
    it
  } else {
    set text(fill: blue)
    it
}})
```

4.6 Showing the entire bibliography

It is sometimes convenient to display the entire bibliography, and not just those references that were actually cited in the current refsection. You can instruct `print-bibliography` to do this using the `show-all` argument:

```
#print-bibliography(
  format-reference: format-reference(),
  show-all: true
)
```

To obtain finer control over the bibliography entries that are shown, you can use the `filter` argument. This function receives a reference dictionary as its argument and returns `true` if this reference should be included in the bibliography and `false` otherwise. For instance, the following call shows all journal articles and nothing else:

```
#print-bibliography(
  format-reference: format-reference(),
  show-all: true,
  filter: reference => reference.entry_type == "article"
)
```

4.7 Continuous numbering

When you typeset a CV, it is sometimes useful to have separate bibliographies for journal articles, papers in conference proceedings, and so on. In this case, you might want to use the *numeric* citation style to number all your references, and you might want to continue counting the papers across the different bibliographies; if you have seven journal papers, you want the first conference paper to be "[8]".

You can achieve this using the `resume-after` parameter of `print-bibliography`. If you pass the number `7` for this parameter, the first entry in the bibliography will be labeled "[8]".

You can automatically continue numbering across multiple bibliographies by passing the `auto` argument to the `resume-after` parameter. If you use `resume-after: auto` for the first bibliography in a refsection, the numbering will start at 1. Each subsequent bibliography in the refsection then determines how many references have already been displayed in this refsection and then starts numbering at this reference count plus 1.

Note that the use of `resume-after: auto` requires an additional iteration of the Typst layout algorithm to converge. It might therefore slow down compilation a little and invite layout convergence issues.

4.8 Highlighting references

The builtin `format-reference` function accepts a parameter `highlight`, which you can use to highlight individual references in the bibliography. The `highlight` function takes a `rendered-reference` as argument; this is a piece of content representing the entire rendered

✷ Emily M. Bender and Alexander Koller. "Climbing towards NLU: On Meaning, Form, and Understanding in the Age of Data". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020.

Figure 7: Highlighting a reference.

bibliography entry, just before it is printed. It also receives the corresponding reference dictionary and the position in the bibliography.

Let's say that you use the `keywords` field in your BibTeX entries to contain the keyword `highlight` if you want to highlight the paper. You can then selectively highlight references like this:

```
#let f-r = format-reference(
  highlight: (rendered, reference, index) => {
   if "highlight" in reference.fields.at("keywords", default: ()) {
      [#text(size: 8pt)[#emoji.star.box] #rendered]
   } else {
      rendered
}})
```

This will place a marker before each reference with the "highlight" keyword, and will leave all other references unchanged.

4.9 Customizing date formatting

Pergamon understands all date types that are exposed by the biblatex crate, including date ranges, BCE dates, and approximate and uncertain dates. Formatting of these dates is controlled by the functions `format-date`, `format-date-range`, `format-date-time`, `format-date-uncertain`, `format-date-approximate`, and `format-date-era`, all of which are parameters of the `format-reference` function. You can pass alternative functions to change the way that dates are rendered in the bibliography.

In addition, Pergamon parses all date fields that BibLaTeX supports: `date`, `eventdate`, `origdate`, and `urldate`. These are represented as part of the reference dictionary as described in Section 6.1. Just like BibLaTeX, Pergamon renders only `date` in its default reference and citation styles. You can modify this behavior through arguments to the reference and citation style. For example, the code below includes the `origdate` if it is defined and renders it as "origdate/date".

```
#let year(reference, field) =
  str(reference.parsed_dates.at(field).start.year)

#let orig-and-pub-year(reference) =
  year(reference, "origdate") + "/" + year(reference, "date")

#let style = authoryear-style(
  citation: (
    format-date: (reference, options) => {
```

```
10        orig-and-pub-year(reference)
11      },
12    ),
13    reference: (
14      format-functions: (
15        "date-with-extradate": (reference, options) => {
16          let extradate = (dev.printfield)(reference, "extradate", options)
17          epsilons(orig-and-pub-year(reference), extradate)
18        },
19      ),
20    ),
21  )
```

5 Caveats

5.1 Special characters in BibTeX fields

By default, Pergamon evaluates the title of a BibTeX entry as Typst code, using Typst's eval function. This is controlled by the `eval-mode` parameter of `format-reference`, the default reference style; see Section 7.3 for details.

This can have unexpected consequences. For instance, consider references whose titles have single quotes that are suitable for LaTeX, such as "`Towards a new ``aesthetic from below''`". Pergamon calls `eval` to evaluate the title as Typst markup, but will choke on the backtick, which it interprets as opening a `raw` element.

To handle such cases, you will need to either modify the BibTeX entry so it avoids special characters that Typst interprets as markup, or you can pass `eval-mode: none` to avoid calling `eval` on titles.

5.2 Layout iterations

In the simplest case, Pergamon requires three iterations of the Typst layout algorithm to converge. First, the cited references in each refsection are collected in a state; second, the bibliography for the refsection is rendered and citation labels are assigned to each reference; third, the citations in the running text are rendered correctly based on these labels.

The number of layout iterations is increased to four if you use the `resume-after: auto` feature in the numeric citation style. This feature requires an accurate count of the references in the first bibliography in order to determine the numbering of the second bibliography. Thus, the labels in the second bibliography only stabil§ize in layout iteration 3, and the citations to these references only stabilize in iteration 4. Be aware that the use of this feature increases the iteration count by one.

Furthermore, the number of layout iterations can occasionally grow to four if a reference in the bibliography occurs close to a page break. Because the exact rendering of a citation into a string changes across layout iterations 1–3, the bibliography itself may shift up or down by a few lines. When this pushes the reference across a page break, Typst needs another layout

iteration to stabilize the page number for its label. This increase in iterations should not be cumulative with the resume-after increase; both together should still be done in four iterations.

6 Data model

Pergamon makes a number of assumptions on the contents of the BibTeX entries. These are typically consistent with those that BibLaTeX makes (see Chapter 2 “Database Guide” in the BibLaTeX documentation), but some of them are worth discussing.

6.1 Dates

Dates can occur in a number of places in the BibTeX entry. The most important one is the publication date of the reference. Pergamon uses the parsed date information that citegeist exposes from the Typst `biblatex` crate, which parses BibLaTeX date fields almost perfectly.

Parsed dates are available in the reference dictionary under `reference.parsed_dates`. The standard keys are `date`, `eventdate`, `origdate`, and `urldate`, when these fields are defined by the bibliography data. The original raw fields remain available under `reference.fields`, but reference and citation styles should use `reference.parsed_dates` for date-sensitive behavior.

A parsed date contains a `kind` field (`"at"`, `"before"`, `"after"`, or `"between"`), boolean `uncertain` and `approximate` markers, and `start` and/or `end` datetime dictionaries. A datetime always has a `year` and may also contain `month`, `day`, and `time`; years may be negative.

Date formatting can be customized through the `format-reference` arguments `format-date`, `format-date-range`, `format-date-time`, `format-date-uncertain`, `format-date-approximate`, and `format-date-era`. These functions receive the date value they format as an explicit argument, followed by the field name and the current `format-reference` options. The top-level `format-date` hook also receives the full reference dictionary. The corresponding default implementations are exported as `default-format-date`, `default-format-date-range`, `default-format-date-time`, `default-format-date-uncertain`, `default-format-date-approximate`, and `default-format-date-era`, so custom hooks can delegate back to the default behavior.

The main hook is `format-date`. It is used for all parsed date fields, including `date`, `eventdate`, `origdate`, and `urldate`; the field being formatted is available as the `field-name` argument:

```
format-date: (date, reference, field-name, options) => ...
```

The lower-level hooks receive only the values they format, the field name, and the options:

- `format-date-range: (start, end, field-name, options) => ...;`
- `format-date-time: (datetime, field-name, options) => ...;`
- `format-date-uncertain: (rendered, field-name, options) => ...;`
- `format-date-approximate: (rendered, field-name, options) => ...;`
- `format-date-era: (year, field-name, options) => ....`

The default `format-date-time` uses `field-name` to choose between two built-in calendar-date styles. For `date`, `eventdate`, and `origdate`, it uses a long human-readable style with localized

month names: `2024-03-14` becomes `14 March 2024`, `2024-03` becomes `March 2024`, and `2024` remains `2024`. For `urldate`, it uses a short numeric style: the same values become `2024-03-14`, `2024-03`, and `2024`. These are defaults of the formatter, not separate option names; override `format-date-time` to use a different style.

For example, this changes only the range separator while keeping all other default date behavior:

```
#let fref = format-reference(
  reference-label: fcite.reference-label,
  format-date-range: (start, end, field-name, options) => {
    let fmt = datetime => options.at("format-date-time")(
      datetime,
      field-name,
      options,
    )
    fmt(start) + " / " + fmt(end)
  },
)
```

To format different date fields differently, override `format-date` and branch on `field-name`. Some BibLaTeX dates may include a time-of-day component, such as `2024-03-14T12:30:45+02:00`. By default, Pergamon prints only the calendar date. Set `show-date-times: true` to include the parsed time of day. When times of day are shown, `show-date-seconds` controls whether seconds are printed, and `show-date-timezones` controls whether UTC markers or numeric timezone offsets are printed.

7 Detailed documentation

Let's now go through the detailed documentation of all the functions and data structures that Pergamon exposes to you.

7.1 Reference dictionaries

The central data structure that Pergamon manages is the *reference dictionary*. This is a dictionary as shown in Figure 8; its purpose is to represent all information about a single bib entry. It is obtained by parsing a BibTeX file using citegeist and then enriching it with some extra information from within Pergamon.

From the perspective of a style developer, the most important part of the reference dictionary is the `fields` part, which contains the fields of the BibTeX entry. In the example, the BibTeX entry defined a number of standard BibTeX fields, such as `author` and `title`; it also defines some lesser-known fields that are standard in BibTeX, such as `keywords`; and it also has extra fields, such as `award`. The reference dictionary makes all of these BibTeX fields available to you.

The keys `entry_type` and `entry_key` at the top of Figure 8 also come from the BibTeX file; they represent the key and entry type of the BibTeX entry. In addition, Pergamon preprocesses the reference dictionary and adds some fields of its own.

```
(
  entry_type: "inproceedings",
  entry_key: "bender20:_climb_nlu",
  parsed_dates: (
    date: (
      kind: "at",
      uncertain: false,
      approximate: false,
      start: (year: 2020),
    ),
  ),
  fields: (
    author: "Emily M. Bender and Alexander Koller",
    award: "Best theme paper",
    booktitle: "Proceedings of the 58th Annual Meeting of the Association
                for Computational Linguistics (ACL)",
    doi: "10.18653/v1/2020.acl-main.463",
    keywords: "highlight",
    title: "Climbing towards NLU: On Meaning, Form, and Understanding
            in the Age of Data",
    year: "2020",
    parsed-author: (
      (given: "Emily M.", family: "Bender"),
      (given: "Alexander", family: "Koller"),
    ),
    sortstr-author: "Bender,Emily M. Koller,Alexander",
    parsed-editor: none,
    parsed-translator: none,
  ),
  label: ("Bender and Koller", "2020"),
  label-repr: "Bender and Koller 2020",
)
```

Figure 8: Example of a reference dictionary.

- The `label` and `label-repr` fields store the output of the `label-generator` call.
- The `parsed-X` fields contain parsed name information. For instance, `parsed-author` represents the outcome of parsing the names in the `author` field. It consists of an array of *name-part dictionaries*, which map the keys `given` and `family` (aka "first" and "last" names) to the parts of that author's name.
- The `sortstr-author` field concatenates the author names, family-name first. It is used when sorting references in a bibliography using the `n` identifier.
- The `parsed_dates` dictionary comes from Citegeist and contains canonical parsed date information.

The preprocessing happens relatively early, so the code in a reference or citation style can rely on the presence of these fields in the reference dictionary.

7.2 Main functions

These are functions implementing the base functionality of Pergamon, such as `cite` and `print-bibliography`.

>> add-bib-resource

Parses BibTeX references and makes them available to Pergamon. Due to architectural limitations in Typst, Pergamon cannot read BibTeX from a file. You will therefore typically call `read` yourself, like this:

```
1 #add-bib-resource(read("bibliography.bib"))
```

You can call `add-bib-resource` multiple times, and this will add the contents of multiple bib files. By default, duplicate bibliography keys are an error, even if they occur in different source files.

Parameters

```
add-bib-resource(
  bibtex-string: str,
  source-id: str none,
  local: bool,
  sentence-case-titles: bool,
  on-duplicate: str
) -> none
```

bibtex-string `str`

A BibTeX string to be parsed.

source-id `str` or `none`

If `source-id` is not `none`, it is added to all references loaded from this BibTeX source under the `source-id` field. This can e.g. be used to filter bibliographies by source id.

For instance, this value of `filter` for `print-bibliography()` will only show the references that were assigned the source id `other.bib`:

```
filter: reference => reference.fields.at("source-id", default: none) ==
"other.bib"
```

Default: `none`

local `bool`

Marks this bibliography resource as *local*. Local bibliographies are only available within the `refsection` in which they are defined; papers in these bibliographies cannot be referenced from outside this refsection.

Pergamon allows you to define the same entry key in both the global and a local bibliography. In case of such collisions, the entry in the local bibliography wins.

Calls to `add-bib-resource` with `local: true` from outside of a refsection are not allowed and will cause an error message.

Default: `false`

sentence-case-titles `bool`

Controls whether Pergamon should convert the titles of all references to sentence case.

If `true`, titles will be converted to sentence case. If `false`, titles will be typeset with the verbatim capitalization specified in the BibTeX entries.

Default: `false`

on-duplicate `str`

Controls how duplicate bibliography keys are handled. The value `"error"` aborts with an error, `"keep-first"` keeps the first entry with a duplicate key, and `"keep-last"` keeps the last entry.

This policy applies both to duplicates within this BibTeX source and to duplicates across multiple calls to `add-bib-resource`.

Default: `"error"`

>> add-category

Adds a category to the given bibliography entries. The primary use of a category is in splitting bibliographies; you could e.g. have one bibliography with highlighted references and another one with the other references by adding some references to a `"highlighted"` category. See the `has-category` function for looking up categories.

Like in BibLaTeX, a category is defined globally for the entire document, not per refsection. Calls to `add-category` should come after the call to `add-bib-resource` that loaded the references to which a category is assigned.

Parameters

```
add-category(
  category: str,
  ..keys: arguments
)
```

category `str`

The category to which the keys should be assigned.

..keys `arguments`

The entry keys in the bibliography that should be assigned to the bibliography.

>> cite

Typesets a citation to the bibliography entry with the given keys. The `cite` function keeps track of what `refsection` we are in and uses that refsection's citation formatter to typeset the citation.

You can pass a single key, `cite(key)`, to typeset a citation of a single reference. Alternatively, you can pass multiple keys, `cite(key1, key2, key3)`, to generate a sequence of citations. Depending on the citation style, this may give you a compact and neat citation, such as "[1, 5]" or "(Author 2020; Other 2021)".

Note that bib keys are always given as strings in Pergamon, e.g. `cite("paper1")`. This is in contrast to Typst's builtin cite function, which expects labels.

You can pass a `form` for finer control over the citation string, depending on what your citation style supports (see Section 3.3). If you do not specify the `form`, its default value of `auto` will generate a default form that depends on the citation style.

You can optionally pass a `prefix` or `suffix` argument to the `cite` call. The authoryear style will place these before or after the main citation, separated by its `prefix-separator` and `suffix-separator` parameters.

Parameters

```
cite(
  ..keys: arguments,
  form: str auto
) -> content
```

..keys `arguments`

The keys of the BibTeX entries you want to cite.

form `str` or `auto`

The citation form.

Default: `auto`

>> citeg

Typesets a citation with the form `"g"`, e.g. “Smith et al.‘s (2020)“. See `cite()` for details.

Parameters

```
citeg(..keys: arguments)
```

..keys `arguments`

The keys of the BibTeX entries you want to cite.

>> citen

Typesets a citation with the form `"n"`, e.g. “Smith et al. 2020”. See `cite()` for details.

Parameters

```
citen(..keys: arguments)
```

..keys `arguments`

The keys of the BibTeX entries you want to cite.

>> citename

Typesets a citation with the form `"name"`, e.g. “Smith et al.”. See `cite()` for details.

Parameters

```
citename(..keys: arguments)
```

..keys `arguments`

The keys of the BibTeX entries you want to cite.

>> citep

Typesets a citation with the form `"p"`, e.g. “(Smith et al. 2020)”. See `cite()` for details.

Parameters

```
citep(..keys: arguments)
```

..keys `arguments`

The keys of the BibTeX entries you want to cite.

>> citet

Typesets a citation with the form `"t"`, e.g. “Smith et al. (2020)”. See `cite()` for details.

Parameters

```
citet(..keys: arguments)
```

..keys `arguments`

The keys of the BibTeX entries you want to cite.

>> citeyear

Typesets a citation with the form `"year"`, e.g. “2020a”. See `cite()` for details.

Parameters

```
citeyear(..keys: arguments)
```

..keys `arguments`

The keys of the BibTeX entries you want to cite.

>> has-category

Checks whether a bibliography entry has been assigned to the given category. The primary purpose of this function is to be used as a `filter` in `print-bibliography`.

The `key` argument can be a string; in this case it is interpreted as the Bibtex key of a bibliography entry, and the function checks whether this key has been assigned to the given `category`.

Alternatively, you can pass a reference dictionary as the `key` argument. In this case, the function will check whether `key.entry_key` has been assigned to the given category.

Parameters

```
has-category(
  key: str dict,
  category: str
) -> bool
```

key `str` or `dict`

The key that should be looked up.

category `str`

The category that should be checked.

>> if-citation

Helper function for rendering the links to a bibliography entry. The first argument is assumed to be a Typst link element, obtained e.g. as the argument of a show rule. If this `link` is a citation pointing to a bibliography entry managed by Pergamon, e.g. generated by Pergamon's `cite` function, the function passes the metadata of this bib entry to the `citation-content` function and returns the content this function generated. Otherwise, the `link` is passed to `other-content` for further processing.

The primary purpose of `if-citation` is to facilitate the definition of show rules. A typical example is the following show rule, which colors references to my own publications green and all others blue.

```
#show link: it => if-citation(it, value => {
   if "Koller" in family-names(value.reference.fields.parsed-author) {
     set text(fill: green)
     it
   } else {
     set text(fill: blue)
     it
}})
```

Parameters

```
if-citation(
  it: link,
  citation-content: function,
  other-content: function
) -> content
```

it `link`

A Typst `link` element.

citation-content `function`

A function that maps the metadata associated with a Pergamon reference to a piece of content. The metadata is a dictionary with keys `reference`, `index`, and `key`. `reference` is a reference dictionary (see Section 7.1), `key` is the key of the bib entry, and `index` is the position in the bibliography.

other-content `function`

A function that maps the `link` to a piece of content. The default argument simply leaves the `link` untouched, permitting other show rules to trigger and render it appropriately.

Default: `x => x`

>> print-bibliography

Prints the bibliography for the `refsection()` in which it is contained. This function cannot be used outside of a refsection.

Parameters

```
print-bibliography(
  format-reference: auto function,
  label-generator: auto function,
  sorting: function str none,
  use-prefix-in-sorting: bool,
  show-all: bool,
  mincrossrefs: int,
  minxrefs: int,
  filter: function,
  grid-style: dictionary,
  title: str content none,
  outlined: bool,
  name-fields: array,
  labelname-fields: array,
  resume-after: int auto,
  reversed: bool
) -> none
```

format-reference `auto` or `function`

The reference style that should be used to render a reference into Typst content.

A reference style is a function that takes two arguments. It takes the position of the reference in the bibliography as a zero-based `int` in the first argument. It takes the reference dictionary for the reference in the second argument.

The function returns an array of contents. The elements of this array will be laid out as the columns of a grid, in the same row, permitting e.g. bibliography layouts with one column for the reference label and one with the reference itself. If only one column is needed (e.g. in the authoryear citation style), `format-reference` should return an array of length one. All calls to `format-reference` should return arrays of the same length.

You can pass the value `auto` instead of a function. In this case, `print-bibliography` will look up the reference formatter from the style bundle that you passed to the `refsection()` that surrounds this call to `print-bibliography`. If you pass `auto` without specifying a `style` for the refsection, a dummy reference style will be used.

Default: `auto`

label-generator `auto` or `function`

The label generator that should be used to generate label information for a reference. This is a function that takes the reference dictionary and the reference's index in the sorted bibliography as input and returns an array `(label, label-repr)`, where `label` can be anything the style finds useful for generating the citations and `label-repr` is a string representation of the label. These string representations are used to detect label collisions, which cause the generation of extradates.

The default implementation simply returns a number that is guaranteed to be unique to each reference. Styles that want to work with `extradate` will almost certainly want to pass a different function here.

The function passed as `label-generator` does not control whether labels are printed in the bibliography in their own separate column; it only computes information for internal use. A style can decide whether it wants to print labels through its `format-reference` function.

Note that `label-repr` *must* be a `str`.

You can pass the value `auto` instead of a function. In this case, `print-bibliography` will look up the label generator from the style bundle that you passed to the `refsection()` that surrounds this call to `print-bibliography`. If you pass `auto` without specifying a `style` for the refsection, the default implementation described above will be used.

Default: `auto`

sorting `function` or `str` or `none`

A function that defines the order in which references are shown in the bibliography. This function takes a reference dictionary as input and returns a value that can be sorted, e.g. a number, a string, or an array of sortable values.

Alternatively, you can specify a BibLaTeX-style sorting string. The following strings are supported:

- `n`: author name (lastname firstname)
- `t`: paper title
- `y`: the year in which the paper was published; write `yd` for descending order
- `d`: the date on which the paper was published; write `dd` for descending order
- `v`: volume, if defined
- `a`: the contents of the `label` field (if defined); for the `alphabetic` style, this amounts to the alphabetic paper key

For instance, `"nydt"` sorts the references first by author name, then by descending year, then by title.

See Section 6.1 for details on how parsed dates are exposed by Citegeist. If a field of the publication date (year, month, day) is missing, it is treated as zero for the purposes of sorting.

If `none` or the string `"none"` is passed as the `sorting` argument, the references are sorted in an arbitrary order. There is currently no reliable support for sorting the references in the order in which they were cited in the document.

Default: `none`

use-prefix-in-sorting `bool`

If `true`, name prefixes such as `van` and `de` are included in the built-in name sorting key `n`. If `false`, names are sorted by family name without the prefix, e.g. `van Gennep` sorts under `Gennep`.

Default: `false`

show-all `bool`

Determines whether the printed bibliography should contains all references from the loaded bibliographies (`true`) or only those that were cited in the current refsection (`false`).

Default: `false`

mincrossrefs `int`

If a bib entry is specified as the parent of another entry through `crossref` links, it will be included in the bibliography when enough of its children have been cited – even if it wasn't cited itself. This option specifies how many children must be cited. It corresponds to `mincrossrefs` in BibLaTeX.

Default: `2`

minxrefs `int`

If a bib entry is specified as the parent of another entry through `xref` links, it will be included in the bibliography when enough of its children have been cited – even if it wasn't cited itself. This option specifies how many children must be cited. It corresponds to `minxrefs` in BibLaTeX.

Default: 2

filter `function`

Filters which references should be included in the printed bibliography. This makes sense only if `show-all` is `true`, otherwise not all your citations will be resolved to bibliography entries. The parameter should be a function that takes a reference dictionary as argument and returns a boolean value. The printed bibliography will contain exactly those references for which the function returned `true`.

Default: `reference => true`

grid-style `dictionary`

A dictionary for styling the grid in which the bibliography is laid out. By default, the grid is laid out with `row-gutter: 1.2em` and `column-gutter: 0.5em`. You can overwrite these values and specify new ones with this argument; the revised style specification will be passed to the `grid` function.

Default: `(:)`

title `str` or `content` or `none`

The title that will be typeset above the bibliography in the document. The string given here will be rendered as a first-level heading without numbering. Pass `none` to suppress the bibliography title.

Default: `"References"`

outlined `bool`

Whether the title of the bibliography should appear in the document's outline.

Default: `true`

name-fields `array`

BibTeX fields that contain names and should be parsed as such. For each `X` in this array, Pergamon will enrich the reference dictionary with a field `parsed-X` that contains an array of name-part dictionaries, such as `("family": "Smith", "given": "John")`. See Section 7.1 for an example.

If the field `X` is not defined in the BibTeX entry, Pergamon will still insert a field `parsed-X`; in this case, it will have the value `none`.

Note that to fully replicate the options `useauthor` / `useeditor` / `usetranslator` in BibLaTeX, you will need to both (a) specify the corresponding option in `format-reference()` and (b) remove the field from the `name-fields` parameter here. This is because `name-fields` is used to determine the reference's `labelname`, long before `format-reference` gets to typeset the reference itself.

Default: `("afterword",`
```
        "annotator",
        "author",
        "bookauthor",
        "commentator",
        "editor",
        "editora",
        "editorb",
        "editorc",
        "foreword",
        "holder",
        "introduction",
        "shortauthor",
        "shorteditor",
        "sortname",
        "translator")
```

labelname-fields `array`

BibTeX fields that will be considered when determining the entry's labelname. The labelname is the field that will be used when generating the labels for the *authoryear* and *alphabetic* citation styles. Labelnames are computed as follows:

- If the BibTeX entry specified a field with the name `labelnamefield`, then the BibTeX field specified under `labelnamefield` is used.
- Otherwise, the first field name in `labelname-fields` that is defined in the BibTeX entry is used.
- If none of these fields is defined, Pergamon will throw an error.

In either of these cases, the name of the BibTeX field from which the labelname is taken is stored in the BibTeX field `labelnamesource`.

Pergamon assumes that the labelname field contains a list of names.

Default: `(`
```
      "shortauthor",
      "author",
      "shorteditor",
```

```
    "editor",
    "translator"
  )
```

resume-after `int` or `auto`

Starts the numbering of entries in this bibliography after the number specified in this argument. Let's say you typeset two bibliographies in your document, and the first one has 15 entries. You can pass `15` in the `resume-after` argument to make the numbering of entries in the second bibliography start at 16.

The `index` parameters of functions like `format-reference` and `format-citation` will receive the sum of resume-after and the actual position in this particular bibliography as an argument. In the example above, the first reference in the second bibliography will be called with index=15 (because the count in the second bibliography is zero-based). The only default citation style that cares about indices is *numeric*.

If you have multiple calls to `print-bibliography` within the same `refsection`, you can pass `auto` to `resume-after` to seamlessly continue the numbering across bibliographies within the same refsection. Note that this requires slightly complex state management, and using the `auto` argument will require Typst to perform four iterations to make the layout converge (rather than three for other uses of Pergamon).

Default: `0`

reversed `bool`

Prints the bibliography in reverse order. This does not change the numbering of the references – e.g. in the numeric style, the first reference in the order of the `sorting` order is still called "[1]". However, if `reversed` is set to `true`, the reference [1] will be the final reference in the bibliography.

Default: `false`

>> refsection

Defines a section of the document with its own bibliography. You need to load a bibliography with the `add-bib-resource()` function in a place that is earlier than the refsection in rendering order.

Each refsection is automatically assigned a unique identifier that distinguishes it from all other refsections in the document. These refsection identifiers are used to generate unique Typst labels for all references in the document, even when they represent the same Bibtex key. Users should make no assumptions about the form of these identifiers beyond their uniqueness.

Note that unlike in Pergamon 0.6.0 and earlier, it is no longer possible to specify the refsection identifier explicitly.

Parameters

```
refsection(
  format-citation: function auto,
  style: dictionary auto,
  doc: content
) -> none
```

format-citation `function` or `auto`

The citation formatter that should be used to generate citation strings within this refsection. It typically comes from a citation style.

A citation formatter is a function that receives an array of *citation specifications* as its first argument, a `form` string as its second argument, and an `options` dictionary as its third argument. It returns the content that is displayed in place of a `cite()` call.

A citation specification is an array `(lbl, reference)`, where `lbl` is a citation label and `reference` is a reference dictionary. The citation formatter is expected to use the information in the reference dictionary to generate the citation and then embed it in a link to the given label (which is anchored by the reference in the bibliography). This might look like this:

```
#link(label(lbl), format(reference))
```

In addition to `(lbl, reference)` pairs, the citation specification array can also contain elements that are strings (i.e. Typst objects of type `str`). This happens in cases where the user cites a paper that does not exist in the bibliography. In this case, the string is the key of the cited paper, and the citation formatter is expected to render an appropriate error message. The builtin styles render the key as "**?key?**"".

The `form` string specifies the exact form in which the citation is rendered; see Section 3.3 for details. This makes the difference e.g. between "Smith et al. (2025)" and "(Smith et al. 2025)".

The `options` dictionary specifies options that control the rendering of the citation in detail. For instance, the *authoryear* style accepts `prefix` and `suffix` arguments. Not every citation style is required to interpret the same options; see the documentation of the citation style for details.

You can pass `auto` in this argument to indicate that you want to use the same citation formatter as in the previous `refsection`. If you pass `auto` to the first refsection in the document, Pergamon will use the dummy citation formatter `(references, form) => [CITATION]`.

See the documentation of the `style` parameter for an alternative way to specify citation formatters.

Default: `auto`

style `dictionary` or `auto`

The style bundle that should be used to typeset citations and bibliographies within this refsection.

A style bundle combines citation style and a reference style in a single dictionary, as explained in Section 3.6. The citation style will be used to format citations within the refsection, and the reference style will be used to typeset the references in all calls to `print-bibliography()` within the refsection.

You will typically specify how to format citations by passing *either* a `format-citation` *or* a `style` argument. The `style` argument is much more convenient, so there is a good chance that this is how most users will do it. However, you are allowed to specify both `format-citation` and `style` arguments; in this case, `format-citation` takes precedence. The `style` will still provide a reference style and label generator to the `print-bibliography` calls.

If you pass `auto` as the `style` argument, the refsection will use the same style as the previous refsection in the document.

Default: `auto`

doc `content`

The section of the document that is to be wrapped in this `refsection`.

7.3 Builtin reference style

Here we explain the builtin reference style.

>> format-reference

The standard reference style. It is modeled after the standard bibliography style of BibLaTeX.

A call to `format-reference` takes a number of options as argument and returns a function that will take arguments `index` and `reference` and return a rendered reference. This function is suitable as an argument to the `format-reference` parameter of `print-bibliography`, and will control how the references in this bibliography are rendered. See the documentation of `print-bibliography` for a more detailed specification of the `format-reference` function in general.

Most of the options of `format-reference` have sensible default values. The one exception is the mandatory named argument `reference-label`, which you obtain from your citation style.

Parameters

```
format-reference(
  reference-label: function,
  highlight: function,
  link-titles: bool,
  print-identifiers: array,
  print-url: bool,
  print-doi: bool,
  print-eprint: bool,
  use-author: bool,
  use-translator: bool,
  use-editor: bool,
  print-date-after-authors: bool,
  eval-mode: str none,
  eval-scope: dictionary,
  list-middle-delim: str content,
  list-end-delim-two: str content,
  list-end-delim-many: str content,
  author-type-delim: str,
  subtitlepunct: str,
  format-journaltitle: function,
  format-title: function,
  format-issuetitle: function,
  format-maintitle: function,
  format-booktitle: function,
  format-fields: dictionary,
  format-functions: dictionary,
  format-parens: function,
  format-brackets: function,
  format-quotes: function,
  format-date: function,
  format-date-range: function,
  format-date-time: function,
  format-date-uncertain: function,
  format-date-approximate: function,
  format-date-era: function,
  show-date-times: bool,
  show-date-seconds: bool,
  show-date-timezones: bool,
  name-format: str dictionary,
  volume-number-separator: str,
  bibeidpunct: str,
  bibpagespunct: str,
  print-isbn: bool,
  bibstring-style: str,
  bibstring: dictionary,
  additional-fields: array none,
  suppress-fields: array dictionary none,
```

```
  period: str array,
  comma: str array,
  maxnames: int,
  minnames: int
)
```

reference-label `function`

The reference labeler that should be used for this bibliography; see Section 3.5 for a detailed explanation.

The reference labeler typically comes from a citation style (e.g. *authoryear*, *numeric*, *alphabetic*).

Unlike the other parameters of `format-reference`, you *must* pass a meaningful argument for this parameter. If you leave it at the default value of `none`, Typst will not even show you a proper error message; it will just say "warning: layout did not converge within 5 attempts".

Default: `none`

highlight `function`

Selectively highlights certain bibliography entries. The parameter is a function that is applied at the final stage of the rendering process, where the whole rest of the entry has already been rendered. This is an opportunity to e.g. mark certain entries in the bibliography by boldfacing them or prepending them with a marker symbol. See Section 4.8 for an example.

The highlighting function accepts arguments `rendered-reference` (`str` or `content` representing the reference as it is printed), `index` (position of the reference in the bibliography), and `reference` (the reference dictionary). It returns `content`. The default implementation simply returns the `rendered-reference` unmodified.

Default: `(rendered-reference, index, reference) => rendered-reference`

link-titles `bool`

If `true`, titles are rendered as hyperlinks pointing to the reference's DOI or URL. When both are defined, the DOI takes precedence.

Default: `true`

print-identifiers `array`

Array of reference identifiers that should be printed at the end of the bibliography entry. The array contains a list of strings; possible values are `"doi"`, `"url"`, and `"eprint"`. For

each bib entry, these values are considered in the order in which they appear in the array, and the first value that is defined as a key in the bib entry is printed.

`print-identifiers` acts as a flexible version of the `print-url`, `print-doi`, and `print-eprint` parameters. For the first `X` in the array that is defined in the bib entry, it effectively sets `print-X` to `true`. The values of the other `print-X` parameters are left unchanged; thus, you could e.g. still force the rendering of `eprint` by setting `print-eprint` to `true`, even if `print-identifiers` matches on the DOI instead.

Default: `()`

print-url `bool`

If `true`, prints the reference's URL at the end of the bibliography entry.

Default: `false`

print-doi `bool`

If `true`, prints the reference's DOI at the end of the bibliograph entry.

Default: `false`

print-eprint `bool`

If `true`, prints the reference's eprint information at the end of the bibliography entry. This could be a reference to arXiv or JSTOR.

Default: `true`

use-author `bool`

If `true`, prints the reference's author if it is defined.

Default: `true`

use-translator `bool`

If `true`, prints the reference's translator if it is defined. See also `name-fields` in `print-bibliography()`.

Default: `true`

use-editor `bool`

If `true`, prints the reference's editor if it is defined. See also `name-fields` in `print-bibliography()`.

Default: `true`

print-date-after-authors `bool`

If `true`, Pergamon will print the date right after the authors, e.g. 'Smith (2020). "A cool paper".' If `false`, Pergamon will follow the normal behavior of BibLaTeX and place the date towards the end of the reference.

Default: `false`

eval-mode `str` or `none`

When Pergamon renders a reference, the title is processed by Typst's eval function. The `eval-mode` argument you specify here is passed as the `mode` argument to `eval`.

The default value of `"markup"` renders the title as if it were ordinary Typst content, typesetting e.g. mathematical expressions correctly.

You can pass `none` to typeset the title literally, without calling `eval`.

Default: `"markup"`

eval-scope `dictionary`

The `scope` argument that is passed to the `eval` call (see `eval-mode`). This allows you to call Typst functions from within the BibTeX entries.

Default: `(:)`

list-middle-delim `str` or `content`

When typesetting lists (e.g. author names), Pergamon will use this delimiter to combine list items before the last one.

Default: `", "`

list-end-delim-two `str` or `content`

When typesetting lists (e.g. author names), Pergamon will use this delimiter to combine the items of lists of length two.

Default: `" and "`

list-end-delim-many `str` or `content`

When typesetting lists (e.g. author names), Pergamon will use this delimiter in lists of length three of more to combine the final item in the list with the rest.

Default: `", and "`

author-type-delim `str`

String that is used to combine the name of an author with the author type, e.g. “Smith, editor”.

Default: `","`

subtitlepunct `str`

String that is used to combine a title with a subtitle.

Default: `"."`

format-journaltitle `function`

Renders the title and subtitle of a journal as content. The default argument typesets it in italics.

Certain titles (title, journaltitle, issuetitle, maintitle, booktitle) can be combined with subtitles (e.g. journalsubtitle). The title and subtitle are joined by `subtitlepunct` to obtain e.g. “Journal title: subtitle”. The subtitlepunct needs to be formatted the same as the title and subtitle, but its formatting cannot be controlled by `format-fields`. This is why Pergamon offers parameters such as `format-journaltitle` to format the entire concatenated title and subtitle.

Default: `it => emph(it)`

format-title `function`

Formats the entry’s title as content. The `title` argument is the already-composed and possible hyperlinked title and subtitle. The purpose of the `format-title` function is to decorate the title as appropriate for the entry type, e.g. by adding quotes or italics. The BibLaTeX equivalent is `printtext[title]`.

The default implementation quotes titles in article-like entries, adds italics in book-like entries, and leaves other titles plain.

Default:
```
(reference, title, options) => {
      let bib-type = reference.entry_type

      if bib-type in ("article", "inbook", "incollection", "inproceedings",
"patent", "thesis", "unpublished") {
        options.at("format-quotes")(title)
      } else if bib-type in ("suppbook", "suppcollection", "suppperiodical",
"misc") {
```

```
      title
    } else {
      emph(title)
    }
  }
```

format-issuetitle `function`

Renders the title of a special issue as content. The default argument typesets it in italics.

See `format-journaltitle` for further explanation.

Default: `it => emph(it)`

format-maintitle `function`

Renders the main title of a multi-volume work as content. The default argument typesets it in italics.

See `format-journaltitle` for further explanation.

Default: `it => emph(it)`

format-booktitle `function`

Renders the title of a book as content. The default argument typesets it in italics.

See `format-journaltitle` for further explanation.

Default: `it => emph(it)`

format-fields `dictionary`

Overrides the way individual fields in the reference are rendered. The argument is a dictionary that maps the names of fields in a BibTeX entry to *extended field formatters*, i.e. functions that compute Typst content and take the following positional parameters:

- `dffmt`: the *default* field formatter, which is applied if no more specific field formatter is specified through `format-fields`.
- `value`: the value of the field in the BibTeX entry.
- `reference`: the reference dictionary, cf. Section 7.1.
- `field`: the name of the field.
- `options`: a dictionary containing all the options that were passed to `format-reference` as arguments.
- `style`: an optional style specification for the field.

If you do not override the rendering of a field, Pergamon uses a default field formatter, i.e. a function that computes content from the `value`, `reference`, `field`, `options`, and `style`

parameters explained above. This default field formatter is passed as the first argument (`dffmt`) to the extended field formatter explained above.

Some examples of how `format-fields` can be used are shown in Section 4.3.

Default: `(:)`

format-functions `dictionary`

Overrides the way that the complex formatting in the reference style is handled.

The default Pergamon reference style relies on a variety of functions with signature `(reference, options) → content` to render pieces of the reference in the bibliography. For instance, the function `driver-article` typesets a BibTeX entry of type `article`.

By passing a dictionary with an entry `function-name: function` in this parameter, you can override the default implementations of these formatting functions. To find the function names that can be overriden, look for functions that start with `with-default` in reference-styles.typ.

The `format-functions` parameter allows you to deeply customize the way Pergamon displays references, without having to implement your own reference style from scratch. It applies to larger spans of content in the references – in contrast to `format-fields`, which affects the formatting of individual BibTeX fields. See Section 3.4 for examples.

Default: `(:)`

format-parens `function`

Wraps text in round brackets. The argument needs to be a function that takes one argument (`str` or `content`) and returns `content`.

It is essential that if the argument is `none`, the function must also return `none`. This can be achieved conveniently with the `nn` function wrapper, see Section 7.6.

Default: `nn(it => [(#it)])`

format-brackets `function`

Wraps text in square brackets. The argument needs to be a function that takes one argument (`str` or `content`) and returns `content`.

It is essential that if the argument is `none`, the function must also return `none`. This can be achieved conveniently with the `nn` function wrapper, see Section 7.6.

Default: `nn(it => [[#it]])`

format-quotes `function`

Wraps text in double quotes. The argument needs to be a function that takes one argument (`str` or `content`) and returns `content`.

It is essential that if the argument is `none`, the function must also return `none`. This can be achieved conveniently with the `nn` function wrapper, see Section 7.6.

Default: `nn(it => ["#it"])`

format-date `function`

Formats publication dates from a parsed date representation (cf. Section 6.1). The function receives `(date, reference, field-name, options)`, where `date` is the parsed date dictionary and `field-name` is `"date"`, `"eventdate"`, `"origdate"`, or `"urldate"`.

The default handles single dates, before/after dates, closed ranges, approximate/uncertain markers, and delegates datetime/range/era details to the lower-level date formatters below.

Example:

```
format-date: (date, reference, field-name, options) => {
  if field-name == "urldate" {
    "accessed " + options.at("format-date-time")(
      date.start,
      field-name,
      options,
    )
  } else {
    default-format-date(date, reference, field-name, options)
  }
}
```

Default: `default-format-date`

format-date-range `function`

Formats a parsed date range. The function receives `(start, end, field-name, options)`, where `start` and/or `end` may be `none` for open ranges.

The default joins closed ranges with an en dash, and uses leading or trailing en dashes for open ranges when this lower-level formatter is called directly with only one endpoint.

Example:

```
format-date-range: (start, end, field-name, options) => {
  let fmt = datetime => options.at("format-date-time")(
    datetime,
    field-name,
    options,
  )
```

```
  fmt(start) + " / " + fmt(end)
}
```

Default: `default-format-date-range`

format-date-time `function`

Formats one parsed datetime, consisting of a date and a time of day. The function receives `(datetime, field-name, options)`.

The default chooses its date style from `field-name`. For most field names (such as `date`), date fields are printed in long form: `14 March 2024`, `March 2024`, or `2024`, depending on whether the month and day were specified in the bib entry. By contrast, `urldate` is printed in short numeric form by default: `2024-03-14`, `2024-03`, or `2024`.

The default respects suppression of `month` and `day` fields; that is, if they are specified in the `suppress-fields` option, they won't be printed. The time of day is printed only if when `show-date-times` is true.

Example:

```
format-date-time: (datetime, field-name, options) => {
  str(datetime.year)
}
```

Default: `default-format-date-time`

format-date-uncertain `function`

Formats an uncertain date after the base date has been rendered. The function receives `(rendered, field-name, options)`, where `rendered` is the base rendering.

The default appends "`?`".

Example:

```
format-date-uncertain: (rendered, field-name, options) => {
  rendered + " (uncertain)"
}
```

Default: `default-format-date-uncertain`

format-date-approximate `function`

Formats an approximate date after the base date has been rendered. The function receives `(rendered, field-name, options)`, where `rendered` is the base rendering.

The default prepends "`ca. `".

Example:

```
format-date-approximate: (rendered, field-name, options) => {
  "about " + rendered
}
```

Default: `default-format-date-approximate`

format-date-era `function`

Formats the year text used by the default human-readable date formatter. The function receives `(year, field-name, options)`, where `year` is the parsed numeric year. This hook is mainly useful for changing how years before 1 CE are displayed.

The default renders BCE years as "`N BCE`".

Example:

```
format-date-era: (year, field-name, options) => {
  if year < 1 {
    str(1 - year) + " BC"
  } else {
    str(year)
  }
}
```

Default: `default-format-date-era`

show-date-times `bool`

Specifies whether the time of day should be included in rendered datetimes. This affects dates such as `2024-03-14T12:30+02:00`; the "12:30" time part is only shown if `show-date-times` is `true`.

Default: `false`

show-date-seconds `bool`

Specifies whether seconds should be included in rendered datetimes. See also `show-date-times`.

Default: `false`

show-date-timezones `bool`

Specifies whether timezone information should be included in rendered datetimes. See also `show-date-times`.

Default: `false`

name-format `str` or `dictionary`

The format in which names (of authors, editors, etc.) are printed. This is an arbitrary string which may contain placeholders such as `{given}`, `{prefix}`, `{family}`, `{suffix}`, `{given-initials}`, and `{prefix-initials}`; these will be replaced by the person's actual name parts. You can use `{g}`, `{p}`, `{f}`, and `{s}` for initials.

Instead of a string, you can also pass a dictionary in this argument. The keys are name types ("author", "editor", etc.), and the values are name format strings as explained above. In this way, you can specify different name formats for different name types. For name types that don't have a key in the dictionary, the default format of `"{given} {prefix} {family} {suffix}"` will be used.

Default: `"{given} {prefix} {family} {suffix}"`

volume-number-separator `str`

Separator symbol for "volume" and "number" fields, e.g. in `@articles`.

Default: `". "`

bibeidpunct `str`

Separator symbol that connects the EID (Scopus Electronic Identifier) from other journal information.

Default: `", "`

bibpagespunct `str`

Separator symbol that connects the "pages" field with related information.

Default: `", "`

print-isbn `bool`

If `true`, prints the ISBN or ISSN of the reference if it is defined.

Default: `false`

bibstring-style `str`

Selects whether the long or short versions of the bibstrings should be used by default. Acceptable values are "long" and "short". See the documentation of `default-long-bibstring` for details.

Default: `"long"`

bibstring `dictionary`

Overrides entries in the bibstring table. The bibstring table is a dictionary that maps language-independent IDs of bibliographic constants (e.g. “in”) to their language-dependent surface forms (such as “In: “ or “edited by”). The ID-form pairs you specify in the `bibstring` argument will overwrite the default entries.

See the documentation for `default-long-bibstring` in Section 7.6 for more information on the bibstring table.

Default: `(:)`

additional-fields `array` or `none`

An array of additional fields which will be printed at the end of each bibliography entry. Fields can be specified either as a string, in which case the field with that name is printed using the reference style’s normal rules. Alternatively, they can be specified as a function `(reference, options) -> content`, in which case the returned content will be printed directly. Instead of an array, you can also pass `none` to indicate that no additional fields need to be printed.

For example, both of these will work:

```
additional-fields: ("award",)

additional-fields: ((reference, options) =>
   ifdef(reference, "award", (:), award => [*#award*]),)
```

Default: `none`

suppress-fields `array` or `dictionary` or `none`

A specification of field names that should not be printed. References are treated as if they do not contain values for these fields, even if the BibTeX file defines them.

You can pass a dictionary that maps entry types to arrays of strings. `("inproceedings": ("editor", "location"))` means that the editor and location fields will not be printed in `inproceedings` references, but may be printed in other entry types.

You can use the special key `"*"` to suppress fields in *all* entry types. A typical usecase is to hide the months and days of the `date` field and typeset only the year: pass `("*": ("month", "day"))` to `suppress-fields`.

If you don’t need fine-grained control over the entry types, you can simply pass an array of strings instead of a dictionary. These field names will be suppressed in all references.

Finally, you can also pass `none` to indicate that no fields should be suppressed.

Default: `none`

period `str` or `array`

Specifies how string or content elements are joined using period symbols. This corresponds roughly (but not precisely) to blocks in BibLaTeX.

If you specify a string here, this string will be used to join the elements. Pergamon will avoid duplicating connector symbols, i.e. it will not print a period if the preceding symbol was already a period.

Alternatively, you can specify an array `(connector, skip-chars)`, where `connector` is the period symbol. The connector is skipped if the preceding character occurs in the string `skip-chars`.

Default: `(".", ".,?!::")`

comma `str` or `array`

Function that joins an arbitrary number of strings or contents with a comma symbol. This corresponds roughly (but not precisely) to units in BibLaTeX.

See the documentation for `period` for details.

Default: `","`

maxnames `int`

Maximum number of names that are displayed in name lists (author, editor, etc.). If the actual number of names exceeds `maxnames`, only the first `maxnames` names are shown and `bibstring.andothers` (“et al.”) is appended.

This parameter is modeled after the `maxnames`/`maxbibnames` option in BibLaTeX.

Default: `9999`

minnames `int`

Minimum number of names that are displayed in name lists (author, editor, etc.). This can be used in conjunction with `maxnames` to create name lists like “Jones, Smith et al.” (minnames = 2, maxnames = 2).

`minnames` trumps `maxnames`: That is, if the name list is at least as long as `minnames`, the reference will show `minnames` names, even if this exceeds `maxnames`. In typical use cases, `minnames` will be less or equal than `maxnames`, so this situation will usually not occur.

This parameter is modeled after the `minnames`/`minbibnames` option in BibLaTeX.

Default: `9999`

7.4 Builtin citation styles

Here we explain the builtin citation styles.

>> format-citation-alphabetic

The *alphabetic* citation style renders citations in a form like "[BK20]". The citation string consists of a sequence of the first letters of the authors' family names. See Figure 3 for an example.

If there are too many authors, the symbol "+" is appended to the citation string to indicate "et al.", e.g. "[YDZ+25]". What constitutes "too many" is controlled by the `maxalphanames` parameter.

If there is only one author, the first few characters of the author name are displayed instead, as in "[Knu90]". The number of characters is controlled by the `labelalpha` parameter.

If more than one reference would receive the same citation string under this policy, the style appends an "extradate" character. For example, if two papers would receive the label "[BDF+20]", then the first one (in the sorting order of the bibliography) will be replaced by "[BDF+20a]" and the second one by "[BDF+20b]".

The function `format-citation-alphabetic` returns a dictionary with keys `format-citation`, `label-generator`, and `reference-label`. You can use the values under these keys as arguments to `refsection`, `print-bibliography`, and `format-reference`, respectively.

Parameters

```
format-citation-alphabetic(
  maxalphanames: int,
  minalphanames: int none,
  labelalpha: int,
  labelalphaothers: str,
  citation-separator: str content,
  format-brackets: function,
  missing-citation: str,
  missing-citation-placeholder: function
)
```

maxalphanames `int`

The maximum number of authors that will be printed in a citation string. If the actual number of authors exceeds this value, the symbol specified under `labelalphaothers` below will be appended to indicate that the author list was truncated.

Default: 3

minalphanames `int` or `none`

The number of author initials to keep when author lists are truncated because their length exceeded `maxalphanames`. With minalphanames = 2, you get "[AB+26]"; with minalphanames = 3, you get "[ABC+26]".

Note that `maxalphanames` controls *whether* long author lists get truncated, and `minalphanames` controls *how many* authors get included in the truncated author list. Therefore, `minalphanames` can never be greater than `maxalphanames`. If you pass a larger value, it will instead be set to `maxalphanames`.

If you pass `none` (the default), `minalphanames` defaults to `maxalphanames`.

Default: `none`

labelalpha `int`

The maximum number of characters that will be printed for single-authored papers.

Default: `3`

labelalphaothers `str`

The "et al" character that is appended if the number of authors exceeds the value of the `maxalphanames` parameter.

Note that this argument really has to be a string, not content, because it is used in the construction of the reference labels.

Default: `"+"`

citation-separator `str` or `content`

The string that separates the author and year in the `p` citation form.

Default: `", "`

format-brackets `function`

Wraps text in square brackets. The argument needs to be a function that takes one argument (`str` or `content`) and returns `content`.

It is essential that if the argument is `none`, the function must also return `none`. This can be achieved conveniently with the `nn` function wrapper, see Section 7.6.

Default: `nn(it => [[#it]])`

missing-citation `str`

Controls how citations to undefined bibliography keys are handled. The value `"error"` aborts with an error; the value `"placeholder"` renders `missing-citation-placeholder`.

Default: `"placeholder"`

missing-citation-placeholder `function`

Renders a placeholder for an undefined citation key when `missing-citation` is `"placeholder"`.

Default: `key => [*?#key?*]`

>> format-citation-authoryear

The *authoryear* citation style renders citations in a form like "(Bender and Koller 2020)". The citation string consists of a sequence of the authors' family names. See Figure 4 for an example.

If a paper has more than two authors, only the first author will be printed, together with the symbol "et al.".

If more than one reference would receive the same citation string under this policy, the style appends an "extradate" character. For example, if two papers would receive the label "(Yao et al. 2025)", then the first one (in the sorting order of the bibliography) will be replaced by "(Yao et al. 2025a)" and the second one by "(Yao et al. 2025b)".

The *authoryear* citation style supports a particularly rich selection of citation forms (see Table 1).

The function `format-citation-authoryear` returns a dictionary with keys `format-citation`, `label-generator`, and `reference-label`. You can use the values under these keys as arguments to `refsection`, `print-bibliography`, and `format-reference`, respectively.

Parameters

```
format-citation-authoryear(
  format-parens: function,
  format-brackets: function,
  author-year-separator: str content,
  citation-separator: str content,
  merge-citations: bool,
  merge-separator: str content,
  format-date: function,
  prefix-separator: str,
  suffix-separator: str,
  list-middle-delim: str content,
  list-end-delim-two: str content,
  list-end-delim-many: str content,
  bibstring: dictionary,
```

```
  bibstring-style: str,
  maxnames: int,
  minnames: int,
  missing-citation: str,
  missing-citation-placeholder: function
)
```

format-parens `function`

Wraps text in round brackets. The argument needs to be a function that takes one argument (`str` or `content`) and returns `content`.

It is essential that if the argument is `none`, the function must also return `none`. This can be achieved conveniently with the `nn` function wrapper, see Section 7.6.

Default: `nn(it => [(#it)])`

format-brackets `function`

Wraps text in square brackets. The argument needs to be a function that takes one argument (`str` or `content`) and returns `content`.

It is essential that if the argument is `none`, the function must also return `none`. This can be achieved conveniently with the `nn` function wrapper, see Section 7.6.

Default: `nn(it => [[#it]])`

author-year-separator `str` or `content`

The string that separates the author and year in the `p` citation form.

Default: `" "`

citation-separator `str` or `content`

Separator symbol to connect the citations for the different keys.

Default: `"; "`

merge-citations `bool`

Whether to merge consecutive citations that share the same author(s) into a single group. When enabled, `#cite("doe2023", "doe2024")` renders as "(Doe 2023, 2024)" instead of "(Doe 2023; Doe 2024)".

Default: `true`

merge-separator `str` or `content`

Separator symbol to connect years within a merged citation group. When multiple consecutive citations share the same author(s), they are merged into a single group (e.g. "Author (2023, 2024)"). This separator is placed between the years within such a group. Only used when `merge-citations` is `true`.

Default: `", "`

format-date `function`

Formats the date component of authoryear citation labels. The function receives `(reference, options)` and should return a string or `none`. The returned string is used both for the displayed citation date and for extradate collision detection. If the function returns `none`, the citation label uses `options.bibstring.nodate`.

The default returns the publication year from `reference.parsed_dates.date`. A style can override this to include more date precision or to combine `origdate` and `date`, e.g. `"1920/2020"`.

Default:
```
(reference, options) => {
    let date = get-date(reference, "date")
    if date != none and date-year(date) != none {
      str(date-year(date))
    } else {
      none
    }
  }
```

prefix-separator `str`

The string that separates the prefix from the citation.

Default: `" "`

suffix-separator `str`

The string that separates the suffix from the citation.

Default: `", "`

list-middle-delim `str` or `content`

When typesetting lists (e.g. author names), Pergamon will use this delimiter to combine list items before the last one.

Will typically match the `list-middle-delim` argument of `format-reference()`.

Default: `", "`

list-end-delim-two `str` or `content`

When typesetting lists (e.g. author names), Pergamon will use this delimiter to combine the items of lists of length two.

Will typically match the `list-end-delim-two` argument of `format-reference()`.

Default: `" and "`

list-end-delim-many `str` or `content`

When typesetting lists (e.g. author names), Pergamon will use this delimiter in lists of length three of more to combine the final item in the list with the rest.

Will typically match the `list-end-delim-many` argument of `format-reference()`.

Default: `", and "`

bibstring `dictionary`

Overrides entries in the bibstring table. The bibstring table is a dictionary that maps language-independent IDs of bibliographic constants (e.g. “in”) to their language-dependent surface forms (such as “In: “ or “edited by”). The ID-form pairs you specify in the `bibstring` argument will overwrite the default entries.

Will typically match the `bibstring` argument of `format-reference()`.

See the documentation for `default-long-bibstring` in Section 7.6 for more information on the bibstring table.

Default: `(:)`

bibstring-style `str`

Selects whether the long or short versions of the bibstrings should be used by default. This controls the rendering of “et al.” and “n.d.”. Acceptable values are “long” and “short”. See the documentation of `default-long-bibstring` for details.

Default: `"short"`

maxnames `int`

Maximum number of names that are displayed in name lists (author, editor, etc.). If the actual number of names exceeds `maxnames`, only the first `maxnames` names are shown and `bibstring.andothers` (“et al.”) is appended.

This parameter is modeled after the `maxnames`/`maxcitenames` option in BibLaTeX. Just like `maxbibnames` is not necessarily the same as `maxcitenames`, the value of `maxnames` here need not match the value of `maxnames` in `format-reference()`.

Default: 2

minnames `int`

Minimum number of names that are displayed in name lists (author, editor, etc.). This can be used in conjunction with `maxnames` to create name lists like "Jones, Smith et al." (minnames = 2, maxnames = 2).

`minnames` trumps `maxnames`: That is, if the name list is at least as long as `minnames`, the reference will show `minnames` names, even if this exceeds `maxnames`. In typical use cases, `minnames` will be less or equal than `maxnames`, so this situation will usually not occur.

This parameter is modeled after the `minnames`/`mincitenames` option in BibLaTeX. Just like `minbibnames` is not necessarily the same as `mincitenames`, the value of `minnames` here need not match the value of `names` in `format-reference()`.

Default: 1

missing-citation `str`

Controls how citations to undefined bibliography keys are handled. The value `"error"` aborts with an error; the value `"placeholder"` renders `missing-citation-placeholder`.

Default: `"placeholder"`

missing-citation-placeholder `function`

Renders a placeholder for an undefined citation key when `missing-citation` is `"placeholder"`.

Default: `key => [*?#key?*]`

>> format-citation-numeric

The *numeric* citation style renders citations in a form like "[1]". The citation string is the position of the reference in the bibliography. See Figure 1 for an example.

The function `format-citation-numeric` returns a dictionary with keys `format-citation`, `label-generator`, and `reference-label`. You can use the values under these keys as arguments to `refsection`, `print-bibliography`, and `format-reference`, respectively.

Note that the `label-generator` of this citation style takes an optional argument `format-string` that controls how the numeric index (1, 2, 3, …) is translated into the actual label. You can pass an oxifmt format string; by default it is `"{}"`, such that the index is rendered without any changes. If you want e.g. labels J01, J02, …, you can pass `label-generator.with(format-string: "J{:02}")` to `print-bibliography` instead. Note that the numeric citation style still

assumes that the labels of all references are different; there is no support for extradates like in the authoryear style. Thus you should probably make sure that `{}` occurs in the format string somewhere.

Parameters

```
format-citation-numeric(
  compact: bool,
  citation-separator: str content,
  compact-separator: str content,
  format-brackets: function,
  missing-citation: str,
  missing-citation-placeholder: function
)
```

compact `bool`

Determines whether sequences of subsequent citation indices should be compacted into a range. If `true`, a citation [1, 2, 3, 5, 6] is instead rendered as [1–3, 5–6].

Note that the citation style does not do anything particular to sort the citation indices within the same citation. The citation [3, 2, 1] will still be rendered as [3, 2, 1] and not as [1–3].

Default: `false`

citation-separator `str` or `content`

The string that separates the different citations when `cite` is called with multiple references (e.g. [1, 3, 15]).

Default: `", "`

compact-separator `str` or `content`

The string that separates the two ends of a range, if the `compact` parameter is set to `true`.

Default: `[--]`

format-brackets `function`

Wraps text in square brackets. The argument needs to be a function that takes one argument (`str` or `content`) and returns `content`.

It is essential that if the argument is `none`, the function must also return `none`. This can be achieved conveniently with the `nn` function wrapper, see Section 7.6.

Default: `nn(it => [[#it]])`

missing-citation `str`

Controls how citations to undefined bibliography keys are handled. The value `"error"` aborts with an error; the value `"placeholder"` renders `missing-citation-placeholder`.

Default: `"placeholder"`

missing-citation-placeholder `function`

Renders a placeholder for an undefined citation key when `missing-citation` is `"placeholder"`.

Default: `key => [*?#key?*]`

7.5 Style bundles

The following functions construct style bundles and provide the three builtin style bundles.

>> alphabetic-style

The *alphabetic* style bundle, which combines the default reference style with the alphabetic citation style.

Parameters

```
alphabetic-style(
  citation: dictionary,
  reference: dictionary
) -> dictionary
```

citation `dictionary`

Dictionary whose entries will be passed as named parameters to the citation style.

Default: `(:)`

reference `dictionary`

Dictionary whose entries will be passed as named parameters to the reference style.

Default: `(:)`

>> authoryear-style

The *authoryear* style bundle, which combines the default reference style with the authoryear citation style.

Parameters

```
authoryear-style(
  citation: dictionary,
  reference: dictionary
) -> dictionary
```

citation `dictionary`

Dictionary whose entries will be passed as named parameters to the citation style.

Default: `(:)`

reference `dictionary`

Dictionary whose entries will be passed as named parameters to the reference style.

Default: `(:)`

>> build-style

Creates a style bundle out of a citation style and a reference style.

Parameters

```
build-style(
  citation-factory: function,
  reference-factory: function,
  citation-parameters: dictionary,
  reference-parameters: dictionary
) -> dictionary
```

citation-factory `function`

A factory for citation styles, i.e. a function which when called will return a citation style.

Citation styles are triples of a label generator, a reference labeler, and a citation formatter, as explained in Section 3.5.

reference-factory `function`

A factory for reference styles, i.e. a function which when called will return a reference style.

Reference styles are functions that typeset reference dictionaries as content, as explained in Section 3.5.

citation-parameters `dictionary`

A dictionary of parameters which will be passed as named arguments to the `citation-factory`.

Default: `(:)`

reference-parameters `dictionary`

A dictionary of parameters which will be passed as named arguments to the `reference-factory`.

Default: `(:)`

>> numeric-style

The *numeric* style bundle, which combines the default reference style with the numeric citation style.

Parameters

```
numeric-style(
  citation: dictionary,
  reference: dictionary
) -> dictionary
```

citation `dictionary`

Dictionary whose entries will be passed as named parameters to the citation style.

Default: `(:)`

reference `dictionary`

Dictionary whose entries will be passed as named parameters to the reference style.

Default: `(:)`

7.6 Utility functions

The following functions may be helpful in the advanced usage and customization of Pergamon.

>> concatenate-names

Concatenates an array of names. If there is only one name, it is returned unmodified. If there are two names, they are concatenated with the value `options.list-end-delim-two` (“and”). If there are three names, they are concatenated with the value `options.list-end-delim-two` (“, “), except the last name is joined with `options.list-end-delim-many` (“, and”).

Parameters

```
concatenate-names(
  names: array,
  options: dictionary,
  maxnames: int,
  minnames: int,
  andothers: bool
) -> str content
```

names `array`

An array of names. Each name can a string or content. If the names are strings, the function will return a string; if at least one name is content, the function will return content.

options `dictionary`

Options that control the concatenation. `concatenate-names` defines reasonable default options for `list-end-delim-two`, `list-end-delim-two`, `list-end-delim-many`, and `bibstring`. You can override these options by passing them in a dictionary here.

Default: `(:)`

maxnames `int`

Maximum number of names that is displayed before the name list is truncated with "et al." See the `maxnames` parameter in `format-reference()` for details.

Default: `2`

minnames `int`

Minimum number of names that is guaranteed to be displayed. See the `minnames` parameter in `format-reference()` for details.

Default: `1`

andothers `bool`

Whether the input name list was explicitly abbreviated with `and others`.

Default: `false`

>> nn

Wraps a function in none-handling code. `nn(func)` is a function that behaves like `func` on arguments that are not `none`, but if the argument is `none`, it simply returns `none`. Only works for functions `func` that have a single argument.

Parameters

```
nn(func) -> function
```

>> family-names

Extracts the list of family names from the list of name-part dictionaries. For instance, the `parsed-author` entry of the example in Figure 8 will be mapped to the array `("Bender", "Koller")`.

If `parsed-names` is `none`, the function returns `none`.

Parameters

```
family-names(parsed-names) -> array none
```

>> format-name

Spells out a name-part dictionary into a string. See the documentation of the `name-format` argument of `format-reference()` for details on the format string.

Parameters

```
format-name(
  name-parts-dict: dictionary,
  name-type: str,
  format: str dictionary
) -> str
```

name-parts-dict `dictionary`

A name-part dictionary

name-type `str`

The type of name as which the name-part dictionary should be formatted. If `format` is a dictionary, `format-name` will look up the format string under this key in `format`.

Default: `"author"`

format `str` or `dictionary`

A format string or dictionary which specifies how the name should be formatted. You can either pass a string, or you can pass a dictionary that maps name types to strings.

Default: `"{given} {prefix} {family} {suffix}"`

>> default-long-bibstring

The default long bibstring table.

A bibstring table maps a number of language-independent identifiers to the strings that will be printed in the bibliography (see the `bibstring` parameter of `format-reference`). Modifying the bibstring table can be useful if you want to localize your bibliography to a language other than English, or if your bibliographic conventions differ from those of the standard BibLaTeX style. For instance, to keep the reference style from printing "In:" with a trailing colon, you could use a bibstring table in which the "in" key maps to "In" rather than "In:".

This default bibstring table is a verbatim copy of the standard English bibstring table in BibLaTeX. It contains only the "long" versions of these entries; so references will be typeset to print e.g. "Jones, editor", rather than "Jones, ed.". You can find this table in the file bibstrings.typ.

>> default-short-bibstring

The default short bibstring table.

It is similar to the "long" bibstring table in `default-long-bibstring`, except that it contains the "short" bibstrings. That is, references will be typeset to print e.g. "Jones, ed." rather than "Jones, editor".

8 Differences from BibLaTeX

I call Pergamon "BibLaTeX *inspired*" because BibLaTeX is a huge library, and I will probably never manage to achieve full feature parity with BibLaTeX. Nonetheless, Pergamon covers a large part of the functionality and configurability of BibLaTeX, and the feature gap closes with each release.

To get a sense of where we stand with respect to supporting BibLaTeX features, you can have a look at biblatex-stresstest.typ. It renders the biblatex-examples.bib from the official BibLaTeX Github repository, augmented with examples for a few additional entry types. You can compare:

- biblatex-stresstest.pdf, the result of compiling biblatex-stresstest.typ with Typst (i.e. the bibliography as rendered by Pergamon);
- stresstest-compiled-with-biblatex.pdf, the result of rendering the same bibliography with BibLaTeX (using this tex file).

Note that a handful of bib entries cause Pergamon to crash; these are in unsupported-biblatex-examples. These all involve the use of `crossref`, `set`, or `label` in bib entries without authors or editors.

8.1 String declarations in BibTeX files

Pergamon supports `@string` declarations in BibTeX files, but it requires that you use a slightly different syntax than in standard BibLaTeX.

In BibLaTeX, you declare strings as follows:

```
@string{anch-ie = {Angew.~Chem. Int.~Ed.}}
@string{cup     = {Cambridge University Press}}
@string{dtv     = {Deutscher Taschenbuch-Verlag}}
```

By contrast, declare them as follows in Pergamon:

```
@string{
  anch-ie = {Angew.~Chem. Int.~Ed.},
  cup     = {Cambridge University Press},
  dtv     = {Deutscher Taschenbuch-Verlag},
}
```

This is because Pergamon uses the Typst Biblatex crate to parse BibTeX files, and that crate requires the syntax variant.

8.2 Known limitations

- Pergamon's date handling follows the parsed date information provided by Citegeist. Non-numeric years are not supported (issue #111).
- Pergamon supports the `author`, `editor`, and `translator` fields, but there is currently no support for `editora` and similar fields (issue #104). Furthermore, when the same person has multiple roles, these are printed separately and not aggregated (issue #103).
- Pergamon currently requires all BibTeX entries to specify an author, editor, or translator; there is no support for the `label` or `shorthand` fields (issue #115).
- `set`, `crossref`, `related`, and `pageref` are not yet supported.

9 Changelog

Changes in 0.8.1 (2026-07-20)

- Bumped to Citegeist 0.3.1 and Typst Biblatex 0.12. Bibliography reading should now be much faster, and error reporting is much improved.
- Name printing is now at BibLaTeX parity: prefixes, suffixes, initials all handled correctly.
- More flexible handling of undefined and duplicated reference keys (thanks to navdeeprana, augustebaum, and Nasenbaer39 for suggestions).

- Multiple small improvements and bugfixes (thanks to mo-mit, cuzbog, and maxnoe for the issues).

Changes in 0.8.0 (2026-03-29)

- Introduced style bundles.
- Words that appear after periods are now systematically uppercased, fixing the most glaringly incorrect visual mistake.
- Leading and trailing whitespace in Bibtex fields is now trimmed.
- In the *alphabetic* style, author names with non-ASCII first characters are now treated correctly (thanks to prawin12345 for the pull request).
- In the *authoryear* style, subsequent citations with the same authors can now be automatically merged (thanks to zouharvi for the suggestion).
- Bumped citegeist dependency to 0.2.2.

Changes in 0.7.2 (2026-01-26)

- Added support for refsection-local bibliographies.
- Improved robustness in Bibtex processing.

Changes in 0.7.1 (2026-01-22)

- Added elementary support for HTML export.
- Bumped to citegeist 0.2.1, which parses Bibtex more robustly and has improved error reporting (thanks to Y.D.X. for the pull request).
- Added the ability to define categories (thanks to maxnoe for the suggestion).
- Added minalphanames option to the *alphabetic* citation style (thanks to DorianRudolph for the pull request).

Changes in 0.7.0 (2025-12-31)

- Revamped the way in which references are collected in each refsection (thanks to bluss and SillyFreak for technical advice).
- Dropped support for `count-bib-entries`, which is no longer needed.
- The *numeric* style now supports custom citation labels (thanks to andreas-bulling for the suggestion).
- The *numeric* style now supports a compact form (thanks to thvdburgt for the suggestion).
- The `print-bibliography` function can now print the references in reverse order (thanks to andreas-bulling for the suggestion).
- Fixed a number of bugs (thanks to ironupiwada, zouharvi).
- Fixed some bugs in the documentation (thanks to thvdburgt).

Changes in 0.6.0 (2025-12-06)

- Reduced the number of iterations until layout convergence from five to three.
- Added the `resume-after: auto` parameter to support continuous numbering of references across bibliographies.
- Fixed a number of bugs.

Changes in 0.5.0 (2025-10-31)

- Implemented the complete set of entry types in Biblatex.
- Languages and countries are now rendered correctly through bibstrings.
- Introduced the `format-function` argument, which allows flexible control over how Pergamon formats larger blocks of references. This permits far-reaching modifications of the standard reference style, without having to reimplement the style from scratch.
- Greatly improved support for contributors who are not the author. For instance, books that have only an editor and not an author are now rendered correctly.
- Added an option to choose between long and short bibstrings.
- Fixed a number of bugs.

Changes in 0.4.0 (2025-10-13)

- Introduced the `format-field` argument, which allows flexible control over how Pergamon formats individual BibTeX fields in the reference.
- Added the `minnames` and `maxnames` arguments to `format-reference` and `format-citation-authoryear`, replicating the BibLaTeX options.
- More flexible control over the bibstring table.
- Cleaned up the formatting of titles that permit subtitles.

Changes in v0.3.2 (2025-10-02)

- Fixed a number of bugs.
- Added the `print-identifiers` parameter to `format-reference`.

Changes in v0.3.1 (2025-09-22)

- Fixed a number of bugs.
- Can now specify different name formats for authors, editors, etc.
- Can now specify prefixes and suffixes in authoryear citations.
- Citing a nonexistent reference key now displays a warning.

Changes in v0.3.0

- Author names are now parsed as in BibLaTeX, using the parser of the biblatex crate.
- Added an `eval-scope` argument to `format-reference` and dropped the `eval-mode` parameter from `print-bibliography`. It is now specified directly on `format-reference`.
- Added `name` and `year` citation forms.
- `print-bibliography` now has a `resume-after` parameter.
- More correct date handling: months can now be suppressed in the bibliography, and the `d` sorting specifier works correctly.

Changes in v0.2.0

- Aggregated citation commands: `#cite("key1", "key2", ...)`.
- Support for `date` and `month` fields (thanks to ironupiwada for contributing code).
- Added `source-id` parameter to `add-bib-resource`.
- The `suppress-fields` option in `format-reference` can now be defined per entry type.

- Default style avoids printing two punctuation symbols in a row.
- Defining multiple references with the same key is now an error.